Deterministic Semantic Topologies: BFT-Safe Embedding Quantization for Autonomous Knowledge Graph Assembly over Merkle-DAGs

Fabrizio Salmi

Independent Researcher

March 2026 — Technical Report / Preprint

Abstract. The AIMP Epistemic Layer (L3) provides BFT-deterministic belief aggregation over Merkle-DAGs, with correlation-aware sensor fusion (v0.3.0) and two-pass Markovian trust propagation (v0.2.0). However, the epistemic knowledge graph — the Supports and Contradicts edges that drive trust flow — must be constructed manually by the application. This paper eliminates that requirement.

We introduce *Deterministic Semantic Topologies*: a protocol extension that automatically generates epistemic edges from 256-bit SimHash embeddings using Hamming distance. Claims carry an optional quantized embedding computed application-side from a protocol-mandated canonical embedding model. At each epoch boundary, the protocol computes pairwise Hamming distances in a deterministic batch (sorted by claim ID), emitting Supports edges for close pairs ($d \leq 30$ bits) and Contradicts edges for distant pairs ($d \geq 200$ bits). Edge strength is a linear integer function of distance.

The design introduces three mechanisms: (1) *embedding versioning*, allowing protocol-level model upgrades without breaking existing claims; (2) a *max-k-nearest cap* that bounds topological edge density to $O(N \cdot k)$, preventing $O(E)$ trust propagation from exploding despite the $O(N^2)$ pairwise scan (rendered negligible by hardware popcount); and (3) a *dead zone* between thresholds that prevents spurious edges from semantically ambiguous pairs.

Hamming distance on 256-bit vectors executes in ~ 1 ns via hardware popcount. For 1,000 claims per epoch, the full batch computation adds < 0.5 ms — negligible compared to trust propagation (59% of the v0.2.0 hot path at 140 µs). All arithmetic is integer-only (XOR + popcount + basis-point scaling). No floats in the protocol core. ZK-circuit-compatible.

The implementation adds a new `semantic_topology` module (200 lines) and two fields to the Claim struct. All 64 existing tests pass unchanged; 17 new tests cover Hamming correctness, edge generation determinism, version isolation, k-cap enforcement, and strength gradients. The implementation is open-source (Rust, MIT license).

1. Introduction

Knowledge graphs in centralized systems — Google Knowledge Graph, Wikidata, DBpedia — rely on curated ETL pipelines to establish relationships between entities. A team of engineers decides that "Paris is-capital-of France" and writes the edge. In decentralized multi-agent systems, there is no such team. Autonomous agents produce claims about the world (sensor readings, LLM inferences, intents), and the system must *discover* the relationships between them without human intervention.

The AIMP Epistemic Layer (L3) [1] provides the machinery to *process* a knowledge graph: two-pass trust propagation, contradiction damping, Sybil-resistant reputation, and correlation-aware discounting [2]. But the graph itself — the Supports and Contradicts edges that drive trust flow — must be supplied by the application. This creates a deployment bottleneck: every AIMP application needs a custom edge-generation policy.

We close this gap with *Deterministic Semantic Topologies*: a protocol extension that automatically assembles the knowledge graph from quantized semantic embeddings. The key insight is that Locality-Sensitive Hashing (SimHash) [3] compresses dense floating-point embeddings into compact binary signatures where semantic distance reduces to Hamming distance — an operation that is integer-only, deterministic, and executes in ~ 1 ns via hardware popcount.

1.1. Contributions

1. **Quantized Semantic Distance.** We define `QuantizedEmbedding([u64; 4])`, a 256-bit SimHash stored as 4 machine words. Hamming distance between two embeddings approximates cosine similarity: $\text{sim} \approx \frac{256-d_H}{256}$. The computation is XOR + popcount, requiring zero floating-point operations in the protocol core.

2. **Autonomous Edge Generation.** At each epoch boundary, the protocol sorts claims by ID and computes pairwise Hamming distances. Pairs below a support threshold produce Supports edges; pairs above a contradiction threshold produce Contradicts edges. The batch is deterministic: two nodes with the same claims produce identical edge sets.

3. **Embedding Versioning.** Claims carry an `embedding_version: u32` field. Only claims with matching versions are compared. This allows the protocol to upgrade its canonical embedding model without invalidating existing claims or breaking CRDT convergence.

4. **Bounded Edge Density.** A per-claim `max_k_nearest` cap strictly bounds the resulting knowledge graph to $O(N \cdot k)$ edges. While the pairwise Hamming scan remains $O(N^2)$ in time, hardware popcount renders this cost negligible (50 ms for 10,000 claims). The topological cap ensures that subsequent Markovian trust propagation — which is $O(E)$ per iteration — remains strictly bounded in time and memory.

1.2. Relationship to Prior Work

This paper builds on the AIMP protocol stack:

- **L1/L2 (v0.1.0)**: Merkle-DAG CRDT with Ed25519 signing [4].
- **L3 (v0.2.0)**: Epistemic Layer with integer log-odds and trust propagation [1].
- **L3 (v0.3.0)**: Correlation-aware aggregation with geometric discounting [2].

SimHash was introduced by Charikar [3] as a dimensionality reduction technique preserving cosine similarity. Locality-Sensitive Hashing (LSH) for approximate nearest neighbors was formalized by Indyk and Motwani [5]. Modern sentence embedding models [6], [7] produce dense vectors that SimHash can compress to binary signatures.

The novelty is not SimHash itself, but its integration into a BFT-deterministic CRDT belief aggregation protocol — where the hash must produce identical results across architectures and the resulting graph must satisfy strict algebraic properties.

2. The Canonical Latent Space

2.1. The Cross-Model Alignment Problem

Different LLM families (Llama, Mistral, GPT) produce embeddings in topologically incompatible latent spaces. A cosine similarity of 0.9 between two Llama-3 embeddings is meaningful; the same cosine similarity between a Llama-3 and a Mistral-7B embedding is not.

Runtime alignment (Procrustes rotation, CCA) requires floating-point linear algebra and produces architecture-dependent results — incompatible with BFT determinism.

2.2. Protocol-Mandated Reference Model

We solve this by separating *thinking* from *communicating*. An agent may use any LLM internally for reasoning. Before emitting a Claim to the network, the agent passes the claim text through a protocol-mandated *canonical embedding model* and computes the SimHash. The protocol specifies:

- The canonical model (e.g., all-MiniLM-L6-v2, 20MB)
- The hyperplane matrix (256 random vectors, deterministically derived from `BLAKE3("AIMP_L3_HYPERPLANES_V" || version)`)
- The embedding version number (`embedding_version: u32`)

L3 never executes the embedding model. It receives pre-computed `[u64; 4]` values and compares them via Hamming distance. The float boundary is on the application side, outside the protocol core.

Crucially, L3 does not require strict hardware determinism from the client-side embedding model. Floating-point variations across GPUs and CPUs (IEEE 754 rounding modes, FMA contraction) may cause the resulting 256-bit SimHash to differ by 1–2 bits for the exact same input string on different hardware. This is precisely why we use continuous Hamming distance thresholds ($d \leq 30$) rather than strict equality ($d = 0$). The topological outcome — the creation of a Supports edge — remains identical and BFT-safe regardless of these micro-variations, fully isolating the deterministic L3 core from hardware-level float discrepancies.

2.3. Embedding Version Isolation

Claims carry an `embedding_version` field. The `AutoEdgeGenerator` skips pairs with mismatched versions:

```
if c1.embedding_version != c2.embedding_version {
    continue;
}
```

This enables protocol-level model upgrades: when the community adopts a new canonical model (v2), new claims carry `embedding_version = 2`. Old claims (v1) continue to function — they just don't generate auto-edges with v2 claims. The knowledge graph naturally partitions by version, and the trust propagation operates correctly on each partition independently.

3. Deterministic Semantic Distance

3.1. SimHash Construction (Application-Side)

The agent computes the 256-bit SimHash as follows:

1. Run claim text through the canonical embedding model, producing a dense vector $\boldsymbol{v} \in \mathbb{R}^d$.
2. For each of 256 hyperplanes $\boldsymbol{h}_i$ (from the protocol's deterministic hyperplane matrix):
   - Compute the dot product $\boldsymbol{v} \cdot \boldsymbol{h}_i$.
   - Set bit i to 1 if the dot product is positive, 0 otherwise.
3. Pack the 256 bits into `[u64; 4]`.

This is the *only* step that involves floating-point arithmetic, and it occurs on the agent side, *before* the claim is signed with Ed25519 and enters the CRDT gossip network.

3.2. Hamming Distance (Protocol-Side)

The protocol computes semantic distance as Hamming distance:

```
pub fn hamming_distance(&self, other: &Self) -> u32 {
    self.0.iter().zip(other.0.iter())
        .map(|(a, b)| (a ^ b).count_ones())
        .sum()
}
```

Properties:

- **Range**: $[0, 256]$. Distance 0 = identical meaning. Distance 256 = perfectly opposite.
- **Approximation**: cosine_similarity $\approx \frac{256-d_H}{256}$ [3].
- **Performance**: XOR + popcount. ~ 1 ns on modern CPUs with hardware `POPCNT` instruction.
- **Determinism**: Integer-only. Identical on ARM64, x86_64, WASM.
- **ZK-compatible**: XOR and popcount are trivially expressible as arithmetic circuits.

3.3. Threshold Policy

Given Hamming distance d between two claims:

Condition	Action	Default
$d \leq T_{\text{support}}$	Emit Supports edge	30 bits
$T_{\text{support}} < d < T_{\text{contradict}}$	No edge (dead zone)	—
$d \geq T_{\text{contradict}}$	Emit Contradicts edge	200 bits

The *dead zone* (31–199 bits) prevents spurious edges from semantically ambiguous pairs. At 128 bits (random chance), no relationship is asserted.

3.4. Edge Strength Function

Edge strength scales linearly with distance in basis points:

Supports ($d \leq T_{\text{support}}$):

$$\text{strength}_{\text{bps}} = \text{clamp}\left(10000 - d \times \frac{9000}{T_{\text{support}}}, \quad 1000, \quad 10000\right)$$

Distance 0 produces strength 10000 (maximum confidence). Distance T_{support} produces strength 1000 (minimum). This gradient feeds into the Markovian flow normalization (v0.2.0): stronger edges carry proportionally more trust.

Contradicts ($d \geq T_{\text{contradict}}$):

$$\text{strength}_{\text{bps}} = \text{clamp}\left(1000 + (d - T_{\text{contradict}}) \times \frac{9000}{256 - T_{\text{contradict}}}, \quad 1000, \quad 10000\right)$$

Distance $T_{\text{contradict}}$ produces strength 1000. Distance 256 (bitwise opposite) produces strength 10000.

4. Automatic Edge Generation Pipeline

4.1. Epoch-Batch Generation

Edge generation is performed as a deterministic batch at each epoch boundary, alongside the existing grid-aligned reduction (v0.3.0).

```
Input:  claims[] in current epoch
Output: auto-generated RawEpistemicEdge[]

1. Filter: keep only claims with embedding != None.
2. Sort by claim ID (BFT determinism).
3. For each pair (i, j) where i < j:
   a. Skip if embedding_version mismatch.
   b. Compute Hamming distance.
   c. If d <= T_support: emit Supports edge with strength(d).
   d. If d >= T_contradict: emit Contradicts edge with strength(d).
   e. Skip if either claim has reached max_k_nearest edges.
4. Return edges (deterministic order: sorted by from_hash, to_hash).
```

4.2. Determinism Proof

Theorem. If two nodes N_1 and N_2 hold the same set of claims C after L2 convergence, they generate identical auto-edge sets.

Proof. (a) L2 convergence guarantees $C_1 = C_2$. (b) The filter (embedding $\neq$ None) is a pure predicate on claim content. (c) The sort is by claim ID (BLAKE3 hash of canonical form), deterministic. (d) The pair enumeration (i, j) with $i < j$ is a deterministic function of the sorted order. (e) Hamming distance is commutative and deterministic (XOR + popcount). (f) Threshold comparison is integer. (g) The k-cap

tracks per-claim counts in a HashMap keyed by claim ID, incremented in sorted-pair order. Therefore the edge set is a pure function of C and the protocol constants. □

4.3. Bounded Edge Density

It is essential to distinguish *computational complexity* from *topological complexity*. The pairwise Hamming scan is $O(N^2)$ in time: every pair is evaluated. However, Hamming distance on 256-bit vectors executes in ~ 1 ns (hardware popcount), so 10,000 claims require ~ 50 ms — a one-time epoch-boundary cost.

The critical bound is *topological*: without capping, N claims produce up to $\frac{N(N-1)}{2}$ edges. With `max_k_nearest = 10`, the graph contains at most $O(N \cdot k)$ edges. The `AutoEdgeGenerator` enforces this by tracking per-claim edge counts and skipping pairs where either claim has reached its cap. Since claims are sorted by ID before the scan, the cap truncates the *same* edges on every node (BFT-safe).

For trust propagation (which is $O(E)$ per iteration), this topological bound is the load-bearing invariant: 10,000 claims with $k = 10$ produce at most 100,000 edges, compared to 50,000,000 without capping.

5. Integration with L3

5.1. Pipeline Order

At each epoch boundary, the complete L3 pipeline executes:

1. Collect all claims in the epoch.
2. **NEW**: Generate auto-edges via `AutoEdgeGenerator`.
3. Build `KnowledgeGraph` from all edges (manual + auto).
4. Run two-pass trust propagation (v0.2.0).
5. Run grid-aligned reduction with correlation discounting (v0.3.0).

Auto-edges are `RawEpistemicEdge` like any manual edge. They enter the knowledge graph through the same `build_from_claims()` path and participate in trust propagation identically.

5.2. Orthogonality with CorrelationCell

`CorrelationCell` (v0.3.0) and `QuantizedEmbedding` (v0.4.0) serve different purposes:

Mechanism	**Question**	**Pipeline Stage**
CorrelationCell	"Are these from the same environment?"	Aggregation (discount)
QuantizedEmbedding	"Are these about the same topic?"	Edge generation
SemanticFingerprint	"Are these the same measurement?"	Reduction (compaction)

A claim can carry all three: a fingerprint for grouping, a cell for discounting, and an embedding for edge generation. They are independent axes of the epistemic pipeline.

The interaction between layers is sequential, not conflicting: the `ExactMatchReducer` compacts claims with identical `SemanticFingerprint` *before* edge generation occurs. Auto-edges are therefore generated exclusively between *distinct* nodes in the DAG — claims that share semantic proximity ($0 < d \leq T_{\text{support}}$) but are not exact duplicates. This prevents the auto-edge generator from creating tautological edges between observations that should have been merged by the Reducer.

5.3. Backward Compatibility

Claims with `embedding: None` produce no auto-edges. They participate in the knowledge graph only through manual edges. All existing v0.2.0 and v0.3.0 behavior is preserved: the `AutoEdgeGenerator` is additive, not replacing.

6. Security Analysis

6.1. Attack 1: Adversarial Embedding Manipulation

Attack. A Byzantine node crafts claim text whose SimHash is artificially close to a target claim, generating fake Supports edges.

Defense (Pre-image Resistance). SimHash is a one-way projection: given a target 256-bit hash, finding text that produces a nearby hash requires brute-force search over the text space. Each attempt requires running the canonical embedding model (∼50 ms on CPU) and computing the SimHash (∼1 µs). Finding a text that flips a specific bit requires $\sim 2^8 = 256$ attempts on average (birthday bound on a single bit). Flipping 30 specific bits to cross the support threshold requires coordinated manipulation — economically expensive compared to the reputation cost of the fake claim (delegation spending bounds Sybil creation to $2 \times$ delegator reputation).

6.2. Attack 2: Threshold Gaming

Attack. Attacker crafts claims at exactly $T_{\text{support}} - 1$ to steal Supports edges from high-reputation claims.

Defense (Dead Zone + Strength Gradient). The dead zone (31–199 bits) is deliberately wide. A claim at distance 29 receives only $\text{strength} = 10000 - 29 \times 300 = 1300$ bps — far weaker than a genuine semantic match at distance 5 ($\text{strength} = 8500$ bps). The trust propagation's Markovian flow divides trust proportionally by strength, so weak auto-edges have minimal influence. Combined with reputation weighting (0-bps Sybils contribute zero), the attack yields negligible epistemic gain.

6.3. Attack 3: $O(N^2)$ Computational DoS

Attack. Attacker floods 10,000 claims to force 50 million Hamming distance computations.

Defense (Hardware Speed + Rate Limiting). Hamming distance on 256-bit vectors is ~ 1 ns (hardware popcount). 50 million comparisons = 50 ms — annoying but not fatal. L2 rate limiting (50 msg/sec per peer) bounds claim inflow. The `max_k_nearest` cap ensures the resulting graph has at most $10000 \times 10 = 100000$ edges, well within trust propagation's capacity.

6.4. Attack 4: Embedding Collision

Attack. Two semantically different claims produce identical SimHash (all 256 bits match).

Defense (Statistical Improbability + Layered Verification). For random inputs, collision probability is 2^{-256}. For adversarial inputs, a SimHash collision means identical projections across 256 random hyperplanes — the texts must be nearly semantically identical in the canonical model's latent space. Even if a collision occurs, the trust propagation weighs the resulting edge by both nodes' reputation and applies contradiction damping from opposing evidence.

7. Evaluation

7.1. Test Suite (17 new tests)

Test	Verifies
`test_hamming_distance_identical`	Distance = 0 for same embedding
`test_hamming_distance_opposite`	Distance = 256 for bitwise NOT
`test_hamming_distance_single_bit`	Distance = 1 for single flip
`test_hamming_distance_symmetric`	$d(A, B) = d(B, A)$
`test_hamming_distance_known_flips`	Exact distance for N flipped bits
`test_hamming_distance_range`	Random embeddings $\approx$ 128 apart
`test_auto_edge_supports`	Close pair $\rightarrow$ Supports
`test_auto_edge_contradicts`	Far pair $\rightarrow$ Contradicts
`test_auto_edge_dead_zone`	Medium distance $\rightarrow$ no edge

`test_auto_edge_deterministic`	Same claims any order → same edges
`test_auto_edge_no_embedding_skipped`	None embedding → skip
`test_auto_edge_max_k_cap`	Edge count bounded per claim
`test_auto_edge_strength_gradient`	Closer = stronger
`test_auto_edge_empty_input`	Empty → no edges
`test_auto_edge_single_claim`	Single → no edges
`test_embedding_version_mismatch`	Different versions → no edges
`test_embedding_version_same`	Same version → edges

All 81 tests pass (64 existing + 17 new).

7.2. Computational Cost

Operation	**Claims**	**Estimated Time**
Hamming distance (single pair)	2	~ 1 ns
Epoch batch (all pairs)	100	~ 5 µs
Epoch batch	1,000	~ 0.5 ms
Epoch batch	10,000	~ 50 ms
Epoch batch	100,000	~ 5 s (LSH needed)

For $N \leq 10000$ (typical epoch size), the batch cost is dominated by the $O\left(\frac{N^2}{2}\right)$ pair enumeration. At ~ 1 ns per Hamming distance, this is acceptable. For $N > 10000$, LSH bucketing (group by SimHash prefix) would reduce to $O(N \cdot \text{bucket}_{\text{size}})$ — deferred to v0.4.1.

7.3. Edge Density Analysis

With default thresholds ($T_{\text{support}} = 30$, $T_{\text{contradict}} = 200$) and random embeddings (expected Hamming distance ≈ 128):

- **Supports probability**: $P(d \leq 30) \approx 10^{-12}$ (negligible for random pairs — only semantically similar claims trigger).
- **Contradicts probability**: $P(d \geq 200) \approx 10^{-12}$ (symmetric).
- **Expected auto-edges per epoch**: proportional to the number of genuinely related claim pairs, not to N^2.

The dead zone captures $> 99.99\%$ of random pairs, ensuring the auto-generated graph is sparse and meaningful.

8. Limitations and Future Work

1. **Canonical model dependency.** The quality of auto-generated edges depends on the canonical embedding model. A poor model produces poor SimHash approximations. Model selection requires empirical evaluation outside the protocol's scope.

2. **Float boundary at application side.** The SimHash construction requires floating-point dot products (embedding × hyperplane). This occurs agent-side, not protocol-side. If an agent uses non-deterministic float arithmetic, its SimHash may differ from other agents'. The defense is reputation: inconsistent embeddings produce inconsistent edges, which are penalized by the trust propagation's contradiction damping.

3. **Dead zone tuning.** The default thresholds (30 / 200) are chosen for general-purpose text similarity. Domain-specific applications (e.g., sensor readings with known noise profiles) may require different thresholds.

4. **No cross-version edges.** Claims with different `embedding_version` never produce auto-edges. During model transitions, the knowledge graph temporarily partitions. A bridge mechanism (cross-version similarity via content hash) is left for future work.

5. **Topologically orphaned claims.** Claims that fall entirely within the dead zone ($30 < d < 200$) across every other claim in the epoch generate zero auto-edges, becoming topologically isolated. This is an epistemologically correct outcome: an observation that is neither corroborated nor contradicted by the network remains in the *uncertain* belief state. Its classification relies entirely on `base_trust` (the author's reputation-weighted confidence), preventing unsupported claims from illegitimately inflating network consensus. Manual edges can still be attached to orphaned claims by application-level policies.

6. **LSH for large epochs.** For $N > 10000$ claims per epoch, the $O(N^2)$ pairwise scan becomes expensive. LSH bucketing [5] would reduce this to $O(N \cdot \text{bucket}_{\text{size}})$ at the cost of missing some cross-bucket pairs.

9. Conclusion

We presented Deterministic Semantic Topologies for the AIMP Epistemic Layer, transforming the knowledge graph from a manually-constructed artifact into an autonomously-assembled structure. The core mechanism — 256-bit SimHash with Hamming distance — is simple, fast (~ 1 ns per comparison), deterministic, and ZK-circuit-compatible.

The significance is architectural: with v0.4.0, thousands of autonomous agents can emit claims into a peer-to-peer CRDT network, and the protocol *automatically discovers* which claims support or contradict each other. Combined with the existing L3 machinery — reputation-weighted trust propagation (v0.2.0), correlation-aware discounting (v0.3.0), and the Merkle-DAG CRDT transport (v0.1.0) — this creates a fully autonomous *Truth Discovery Engine* that operates without coordination, without floats, and without trust.

Bibliography

[1] F. Salmi, "Byzantine-Tolerant Belief Aggregation over Merkle-DAGs: A Deterministic Epistemic Layer for Decentralized AI," 2026.

[2] F. Salmi, "Correlation-Aware Belief Aggregation: Deterministic Spatial-Temporal Discounting for Byzantine-Tolerant Sensor Fusion over Merkle-DAGs," 2026. [Online]. Available: https://www.researchgate.net/publication/403250288

[3] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," in *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*, 2002.

[4] F. Salmi, "AIMP: AI Mesh Protocol—Design and Evaluation of a Serverless Merkle-CRDT Protocol for Edge Agent Synchronization," 2026. [Online]. Available: https://www.researchgate.net/publication/403127328

[5] P. Indyk and R. Motwani, "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, 1998.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[7] L. Wang *et al.*, "Text Embeddings by Weakly-Supervised Contrastive Pre-training," 2022. [Online]. Available: https://arxiv.org/abs/2212.03533